EPFL - Computer Science

COM-407 Advanced Networks

Lecture 08 — 07/11/2025§

Author: Jonathan Dutoit

Professor: Pavlos Nikopoulous

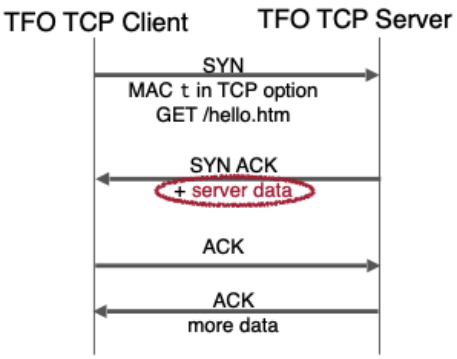
Transport Layer Continued

TCP Fast Open (TFO)

Why? Avoid latency of 3-way handshake when re-opening repeated connections

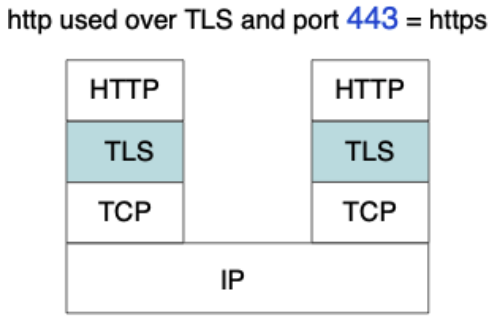
How?

- Server includes in *TCP options* of SYN-ACK a *message authentication code (MAC)*: t = `crypto_hash(k, c)`
  - k is a secret, unknown to client
  - c is the client's IP address
- Client caches t
- Uses it when opening next connection
  - Client sends back t within SYN packet
  - After receiving SYN and t, server knows that "this client has connected earlier"
  - Server can send *data within SYN-ACK*



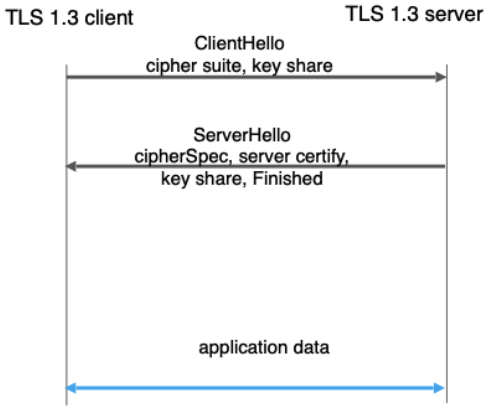
5. Secure Transport

- TCP is not secure
- can be secured with a security sub-layer:
  - → Transport Layer Security (TLS)
- TLS adds to TCP: confidentiality (data is encrypted) and authentication (data is protected against forgery)



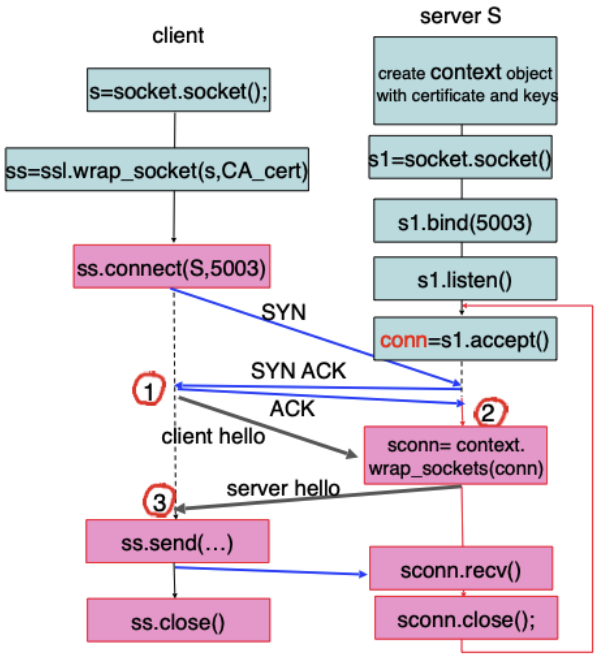
TLS 1.3

- does not support RSA key exchange
- Communicating parties:
  - agree on cryptographic suites
  - exchange cryptographic parameters to compute session keys (∼ "Diffie-Hellman" approach)
- needs *1-RTT handshake* before sending application data



TLS Sockets

TLS sockets (also called SSL sockets) are transformed/"*wrapped*" sockets.



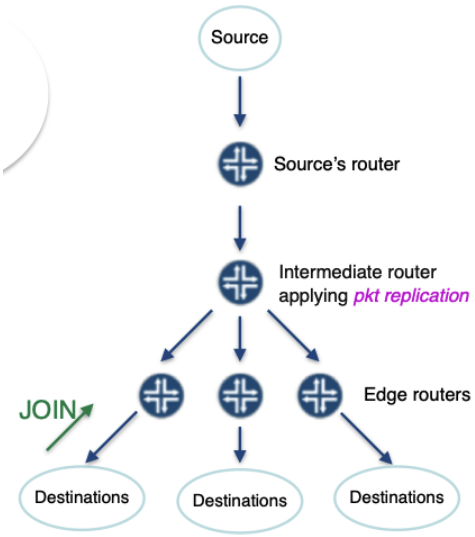
With https/TLS1.3, how many RTTs are required before data transfer can occur?

A. 1
B. 2
C. 3
D. 4
E. 0
F. I don't know

Answer: B. 1 RTT for opening the TCP connection and 1 RTT (at least) for establishing the TLS session

IP Multicast

- Recall:
  - Unicast = source sends data to a *unicast* IP address, and a *single* destination receives it
  - Multicast = source sends data to a *multicast* IP address, but *multiple* destination receive it
- Multicast delivers the same data to multiple recipients *without* sending multiple copies from the source
  → Traffic is *replicated* at the *network layer*
- Used only:
  - with *UDP* (cannot be done with TCP!)
  - within a *single domain* (= network belonging to a single administrative entity, composed of multiple subnets)

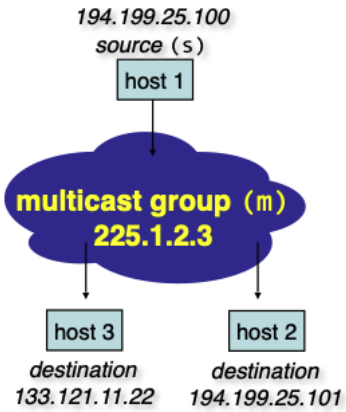


Typical process

1. Destination *subscribe to* a multicast group by sending *join* messages to their gateway routers
   - IGMP (Internet Group Management Protocol, in IPv4)
   - MLD (Multicast Listener Discovery, in IPv6)
2. Routers either build distribution tree (PIM) or use a *tunneling* approach (BIER)
3. Source sends packets to multicast address
4. Intermediate IP routers between source and destinations *replicate traffic*

Multicast addresses and groups

- Reserved address spaces:
  - IPv4: **224.0.0.0/4** (i.e. 224.0.0.0 to 239.255.255.255)
  - IPv6: **ff00::/8**, bits 13-16 determine the "scope":
    ff02/16 = same subnet,
    ff05/16 = same domain



- *Any Source Multicast* (ASM) group:
  - identified by the multicast address;
  - *any* source can send to this group

- *Source Specific Multicast* (SSM) group:
  - identified by `(s, m)`
    where ***m*** is a multicast address and ***s*** is a (unicast) source IP address;
  - *only* ***s*** can send to this group; OS discards packets from irrelevant sources
  - default SSM addresses: 232.0.0.0/8, ff3x::/96 (x=scope bits; e.g., ff35::/96 = site-local)

PIM: Protocol Independent Multicast

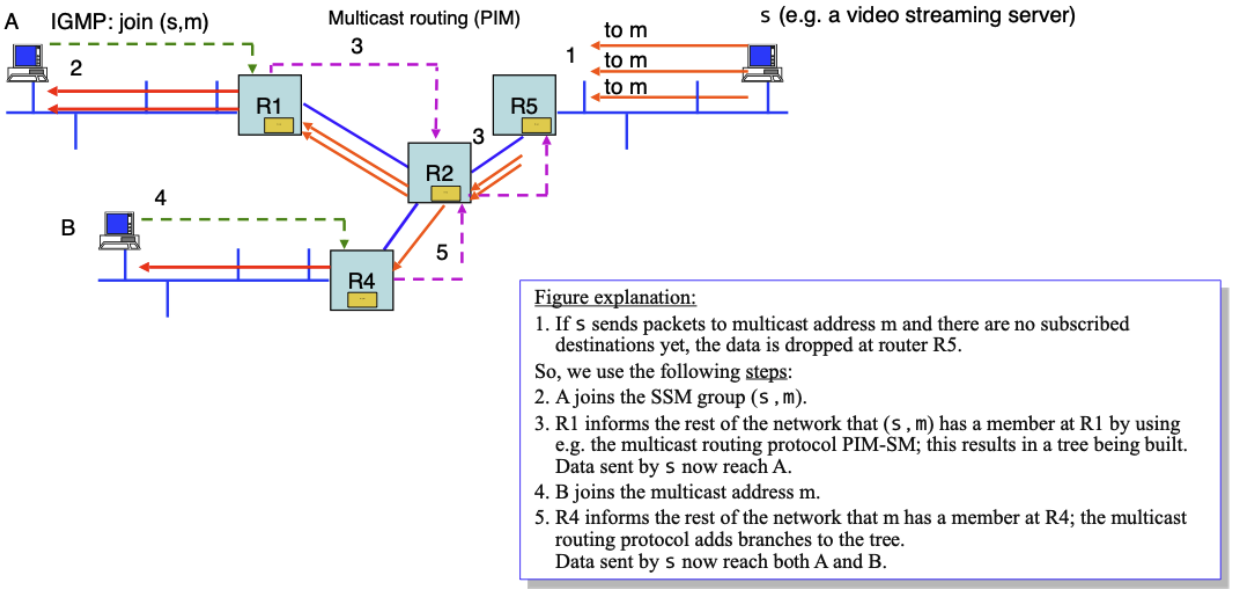


Figure explanation:

1. If s sends packets to multicast address m and there are no subscribed destinations yet, the data is dropped at router R5.

So, we use the following steps:

2. A joins the SSM group (s, m).
3. R1 informs the rest of the network that (s, m) has a member at R1 by using e.g. the multicast routing protocol PIM-SM; this results in a tree being built. Data sent by s now reach A.
4. B joins the multicast address m.
5. R4 informs the rest of the network that m has a member at R4; the multicast routing protocol adds branches to the tree. Data sent by s now reach both A and B.

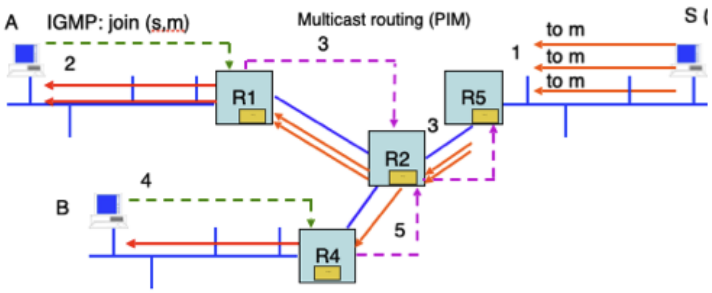
PIM supports ASM and SSM. Exists in 2 versions:

- PIM-DM (*Dense* Mode): heavy use of broadcast, used only in small networks

- PIM-SM (*Sparse* Mode): widespread, used e.g., for TV distribution
  - If used for SSM, PIM-SM uses *reverse path forwarding (RPF)*
  - This creates the distribution tree *on the fly*

PIM routers keep per-(multicast)-flow state

In addition to longest prefix match, PIM routers keep state for multicast groups and apply *exact-match* lookup.



Multicast *state information* is dynamically kept in router for every known multicast group:

- *(s, m) or (*, m)* → id of group
- *valid incoming interfaces* → for security
- *outgoing interfaces* → this is the forwarding info
- *other information required by multicast routing protocol*

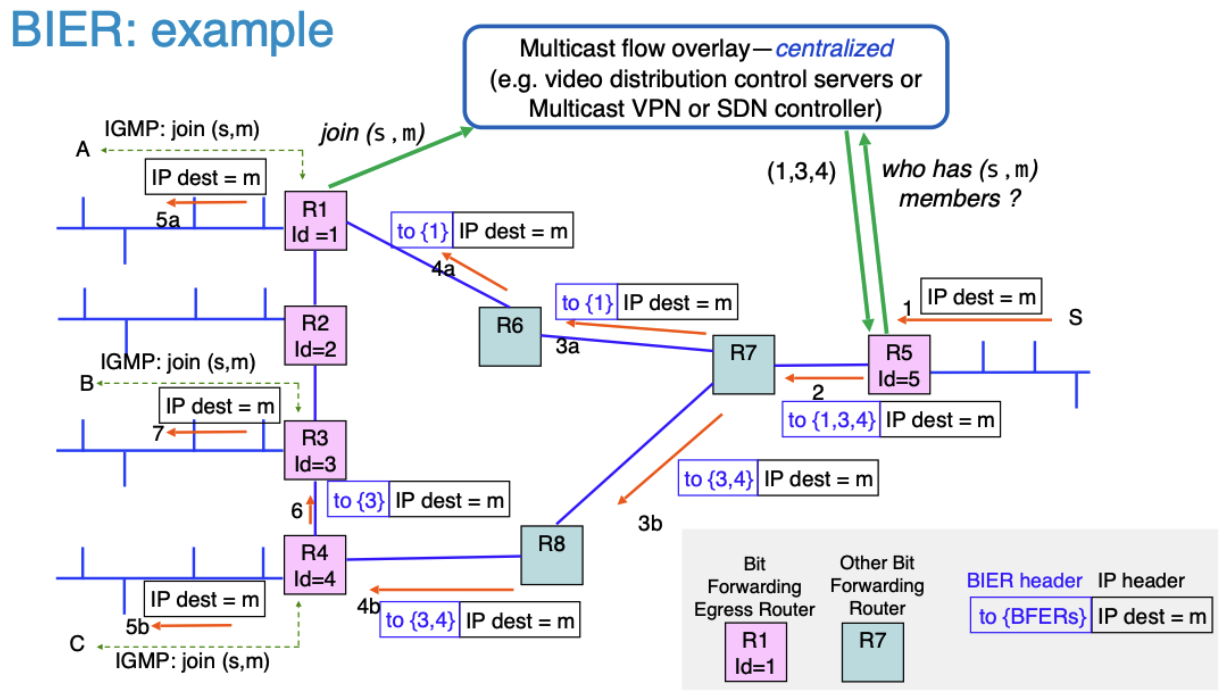
BIER (Bit Index Explicit Replication)

Why?

Per-flow state is *dynamic* (depends on who listens to the group) and *cannot* be aggregated based on prefix
→ *Scalability issues* at backbone routers; state info may be significantly large.

What?

- Keep ***no*** state at backbone routers; put forwarding info *inside* packets (∼tunneling)
  - Every multicast packet has an *extension header* carrying the list of destination routers (a.k.a. Bit-Forwarding Egress Routers, BFERs)
  - Routers use packet's header and their forwarding table to multicast packets:
    - for *all* BFERs in header reached through the same next hop, only a *single* copy is create
    - in this copy, the BIER header is modified accordingly

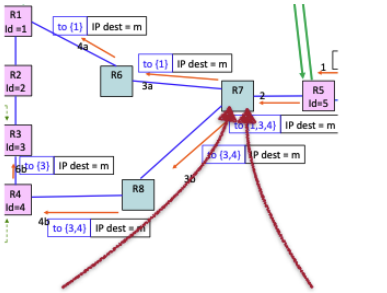


BIER: Packet replication and forwarding

- For each destination BFER, BIER router pre-computes (based on the IP forwarding table) a "forwarding bit mask" that indicates the set of destination BFERs that are reached by the same next-hop

- To forwarding a packet, with destination set S, BIER router runs:

  1. `Send a copy to 1st destination in S with destination set =` $S \cap S_1$`, where` S_1 `is the forwarding bit mask of 1st destination.`
  2. `If` $S \backslash S_1 \neq \emptyset$`, duplicate the packet but with destination` $S \backslash S_1$ `and goto 1; else break.`



BIER Index Forwarding Table at R7

Dest. BFER	Forwarding bit mask	Next-Hop
1	{1,2}	R6
2	{1,2}	R6
3	{3,4}	R8
4	{3,4}	R8
5	{5}	R5

IP Forwarding Table at R7

Dest. IP addr	Next-Hop
R1	R6
R2	R6
R3	R8
R4	R8
R5	R5

Example at router r7
R7 receives packet with destination set `S = {1,3,4}`

1. First destination in S is 1, R7 uses Bier Index Forwarding table and finds that the forwarding bit mask for destination 1 is $S_1 = \{1, 2\}$: sends a copy of packet to next-hop (R6) with destination $S' = S \cap S_1 = \{1\}$
2. $S'' = S \backslash \{1, 2\} = \{3, 4\}$ is not empty; R7 duplicates packet with destination S'' and applies the same: first destination is 3, so R7 sends a copy to next-hop (R8) with destination set $\{3, 4\}$
3. Now $\{3, 4\} \backslash \{3, 4\} = \emptyset$, so it breaks.

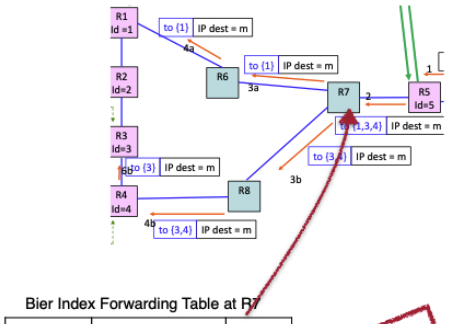
How to optimize processing at BIER routers?

- BIER routers encode the forwarding bit masks and destination sets into *bitstrings*. E.g. with 5 BFERs:
  - Destination BFERs $= \{1, 3, 4\} \rightarrow$ bitstring $= 01101$
  - Destination BFERs $= \{3, 4\} \rightarrow$ bitstring $= 01100$
  - Destination BFERs $= \{1\} \rightarrow$ bitstring $= 00001$
- Set intersection becomes a *bitwise AND with mask*
- Set difference becomes a *bitwise AND with the bit-inverted mask*
- More complicated mechanisms exit, if the number of BFERs is large

BIER - recap

- Routers have 2 tables:
  - Unicast IP forwarding table (longest prefix match)
  - Bit Index Forwarding table is automatically derived from unicast IP forwarding table (exact match); stores no per-flow state
- apply *bistring processing* using a
- Bit Forwarding Ingress Router (BFIR, such as R5) caches mappings of multicast addresses to BIER headers
  - needs to access an *out of band, centralized entity*;
  - this is the *only* (dynamic) per-flow state kept in BIER
- Multicast packets are forwarded based on BIER header, destination IP address is not used (∼*tunneling*)

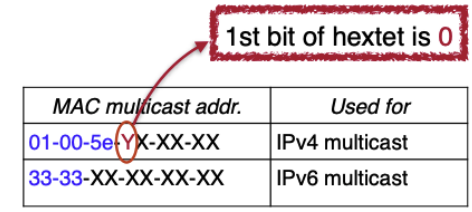
Multiple BIER domains can be interconnected by BFIR-BFER interconnection



Is there Multicast ARP (address resolution protocol)?

- *No*, multicast MAC address is *algorithmically* derived from multicast IP adress:
  - Last 23 bits of IPv4 multicast address are used in MAC addresses
  - Last 32 bits of IPv6 multicast address are used in MAC addresses

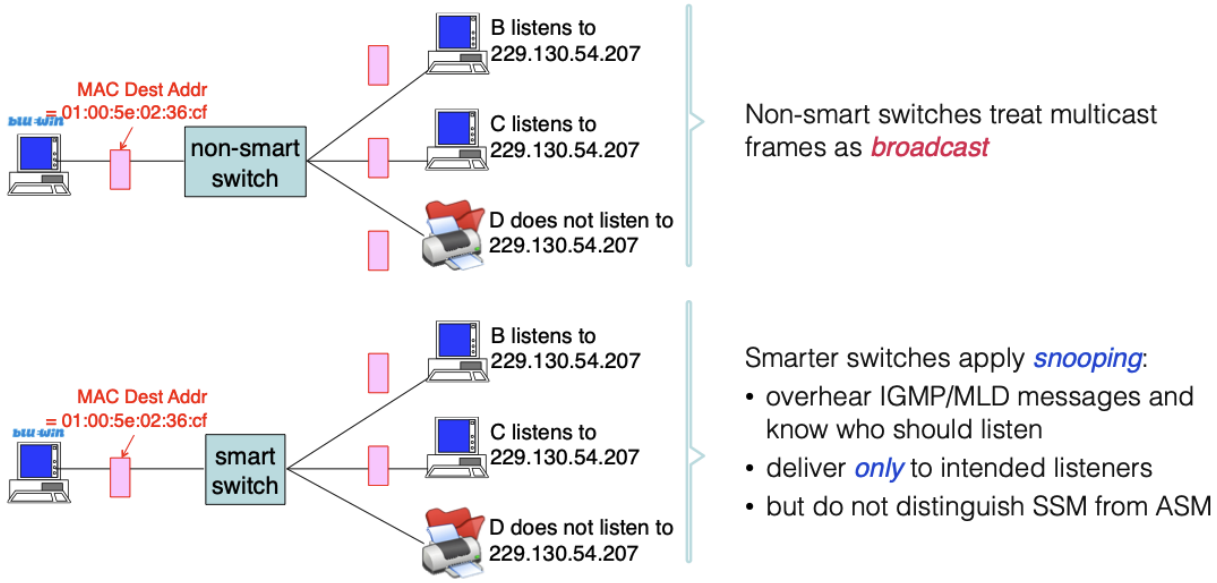
- Notes:
  - Multicast MAC depends only on multicast IP address m, not on source address s, even if m is an SSM address
  - Several multicast IP addresses may yield the same MAC
    - packets received unnecessarily at the MAC layer are removed by the OS; hopefully this happens rarely



MAC multicast addr.	*Used for*
01-00-5e-YX-XX-XX	IPv4 multicast
33-33-XX-XX-XX-XX	IPv6 multicast

IP dest address	229.130.54.207
IP dest address (hexa)	e5-82-36-cf
IP dest address (bin)	...-10000010-...
Keep last 23 bits (bin)	...-00000010-...
Keep last 23 bits (hexa)	02-36-cf
MAC address	01-00-5e-02-36-cf

MAC Multicast: how do switches handle multicast frames?



Security of IP Multicast

IP multicast w/ or w/o BIER makes attacks easier (e.g., *Denial of Service*, witty worm). Mitigations: limit multicast rate and number of groups; control which multicast group is allowed (*access lists*)

SSM is safer because routers and destination can reject misbehaving sources

IGMP/MLD is not secure and has the same problems as ARP/NDP. Mitigated by same mechanisms: sniffing switches observe all traffic and implement *access control*

Multicast-capable networks must deploy exhaustive *filtering* and *monitoring* tools to limit potential damage.

Multicast in Practice

- Multicast is good for sources: one packet sent for n destinations – replication is done $O(log(n))$ times
- Multicast is not supported everywhere, but is used in:
  - Internet TV distribution (PIM-SM / BIER)
  - EPFL and other academic networks (PIM-SM)
  - Data Center Virtualization Services (BIER)
  - Some corporate networks for news, sensor streaming, time synchronization, large video-conferences, etcs...
  - Industrial networks (smart grids, factory automation)
- Works only with UDP, not with TCP (not easy to handle TCP acks from multiple receivers, the goal of multicast would be defeated) → need to handle errors, usually via redundancy (extra traffic)